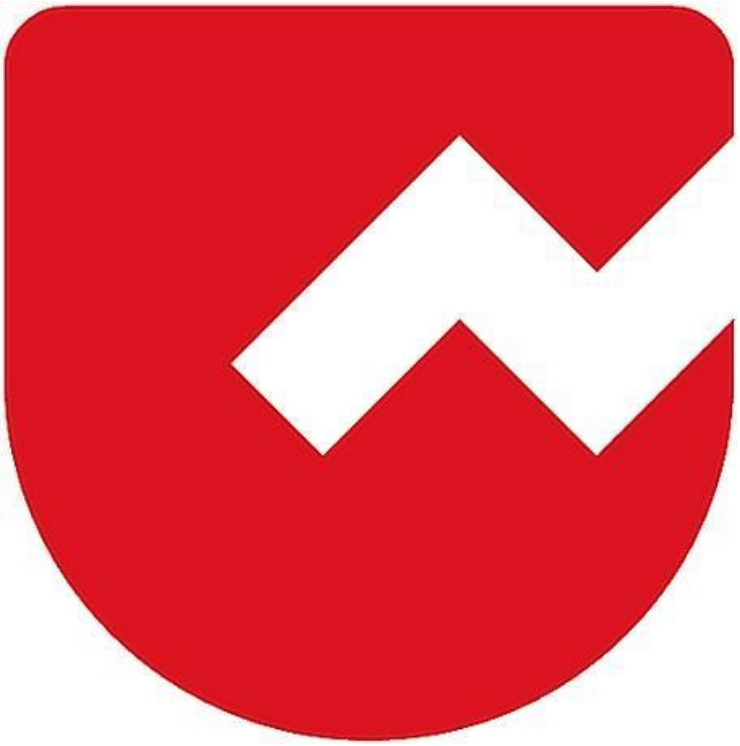
Zoho Schools for Graduate Studies



Notes

Interface

Define Interface?

- An interface defines a set of abstract methods (methods without a body) which a class must implement, if it chooses to implement the interface.
- It can contain only those concrete methods that are marked as default or static or private. Otherwise, an interface cannot have a normal concrete method.
- It can have constants.

```
// InterfaceDemo

        public interface Payment {
            void pay();
            public abstract void checkBalance();
        }

        abstract public class UPI implements Payment {
            public void checkBalance() {
                System.out.println("Checking the Balance");
            }
        }
```

```
public class GooglePay extends UPI {
    public void pay() {
        System.out.println("Payment is made through GPay!");
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Payment g = new GooglePay();
        g.checkBalance();
        g.pay();
    }
}
```

Notes:

1. Interfaces are implicitly abstract. Hence they cannot be instantiated. Explicit declaration of the abstract modifier is optional.
2. Interfaces are assumed to have either public or default access level. Otherwise, they cannot be marked as private, protected, or final.
3. Constants declared inside an interface must be initialized at the place of declaration.

4. Abstract methods inside an interface cannot be marked as static or final or protected or private.
5. The first concrete subclass should ensure that the implementations for all the Inherited abstract methods are available.
6. Interfaces can be implemented by any class from any inheritance tree.
7. Method declarations inside an interface are implicitly abstract and public. Explicit declaration of these modifiers are optional.
8. Constants declared inside an interface are implicitly public, static, and final. Explicit declarations of these modifiers are optional in any combination.
9. An interface can inherited (extend) multiple interfaces.
10. An interface cannot implement another interface or class.
11. An interface cannot extend another class.
12. A class can extend only one another class except Object class. i.e. a class cannot inherit multiple classes.

13. A class can implement multiple interfaces.

14. A class cannot extend another interface.

15. A class cannot implement another class.

16. If a class extends another class and also implements an interface, then the keyword ‘extends’ must appear prior to the keyword ‘implements’.

17. A class implementing an interface can itself be abstract.

18. An abstract implementing class need not have to implement all the interface methods.

19. A legal (concrete subclass) non-abstract implementing class should ensure that the implementations for all the inherited abstract methods are available.

20. The legal non-abstract implementing class must follow all the legal override rules for the methods it implements.

21. Default methods can only be defined inside an interface and cannot be defined inside a class.

22. Default method is a way to provide default implementation for any abstract methods by an interface.

23. Static methods defined in an interface are not inherited by any class that implements the interface.

24. Interface reference names can be used polymorphically, i.e., an object type of the class that implements the interface can be assigned to the interface reference name.

What is a marker interface?

- An empty interface is called as a marker interface.
- Example: Cloneable, Serializable